Metody Inteligencji Obliczeniowej

Sprawozdanie 5

Wydział Fizyki i Informatyki Stosowanej

Mikhail Shupliakou

23.04.2026

Spis treści

1 Wstęp **3**

2 Zadanie 1 **3**

2.1 Opis zadania 3
2.2 Przygotowanie danych 3
2.3 Projektowanie architektur sieci (CNN) 3
2.4 Proces uczenia 4
2.5 Ewaluacja i wizualizacja wyników 4
2.6 Analiza krzywych uczenia (funkcji strat) 4
2.7 Macierze pomyłek 7
2.8 Przykłady błędów klasyfikacji 9

3 Zadanie 2 **10**

3.1 Opis zadania: 10
3.2 Proces realizacji eksperymentu (CPU vs GPU) 10
3.3 Wyniki i wnioski z porównania CPU i GPU 11

Sieci konwolucyjne

1 Wstęp

Konwolucyjne sieci neuronowe (ang. *Convolutional Neural Networks*, CNN) to specjalny rodzaj sztucznych sieci neuronowych, zoptymalizowany do przetwarzania danych o strukturze siatki, takich jak obrazy. Dzięki zdolności do automatycznego i hierarchicznego wyodrębniania cech przestrzennych za pomocą operacji splotu, sieci te stanowią obecnie fundament współczesnego widzenia komputerowego (ang. *computer vision*).

Niniejsze sprawozdanie przedstawia proces projektowania, uczenia oraz ewaluacji architektur konwolucyjnych na przykładzie zadania klasyfikacji obrazów ze zbioru danych Fashion MNIST. W ramach przeprowadzonych eksperymentów przeanalizowano skuteczność trzech różnych wariantów sieci, wykorzystując do oceny macierze pomyłek oraz wykresy funkcji strat. Ponadto porównano czas uczenia modelu z wykorzystaniem procesorów CPU i GPU.

2 Zadanie 1

2.1 Opis zadania

W ramach zadania należy pobrać zbiór danych Fashion MNIST przy użyciu biblioteki `torchvision` i zaprojektować konwolucyjną sieć neuronową (CNN) do klasyfikacji zawartych w nim obrazów. Wymagane jest zaimplementowanie i przetestowanie co najmniej trzech różnych architektur sieci. Dla każdego z modeli należy wygenerować wykres funkcji strat w trakcie uczenia, wyznaczyć macierz pomyłek (ang. *confusion matrix*) oraz zaprezentować przynajmniej jeden przykład błędnie zaklasyfikowanego obrazu.

2.2 Przygotowanie danych

Zbiór danych Fashion MNIST został pobrany za pomocą modułu `torchvision.datasets`. Przed podaniem danych do sieci, zastosowano transformacje (moduł `transforms`): konwersję obrazów do postaci tensorów (`ToTensor`) oraz normalizację wartości pikseli ze średnią 0.5 i odchyleniem standardowym 0.5. Następnie utworzono obiekty `DataLoader` dla zbioru treningowego i testowego z rozmiarem wsadu (ang. *batch size*) równym 64. Dodatkowo zdefiniowano tekstowe etykiety klas, co ułatwiło późniejszą analizę wyników.

2.3 Projektowanie architektur sieci (CNN)

Zaprojektowano trzy warianty konwolucyjnych sieci neuronowych w celu porównania ich skuteczności:

- **CNN1:** Najprostsza architektura, składająca się z jednej warstwy konwolucyjnej (16 filtrów o rozmiarze 3x3), warstwy łączącej (*Max Pooling*) oraz jednej warstwy w pełni połączonej (ang. *fully connected*).

- **CNN2:** Bardziej złożony model, zawierający dwie warstwy konwolucyjne (odpowiednio 32 i 64 filtry), warstwę łączącą oraz dwie warstwy w pełni połączone (128 ukrytych neuronów).
- **CNN3:** Architektura bazująca na modelach poprzednich, w której utrzymano dwie warstwy konwolucyjne, ale zwiększono pojemność sieci poprzez rozszerzenie pierwszej warstwy w pełni połączonej do 256 neuronów.

2.4 Proces uczenia

Do optymalizacji wag we wszystkich modelach wykorzystano algorytm Adam ze współczynnikiem uczenia (ang. *learning rate*) wynoszącym 0.001. Jako funkcję strat wybrano entropię krzyżową (`CrossEntropyLoss`). Uczenie przeprowadzono na przestrzeni 5 epok. Przed rozpoczęciem treningu modele były każdorazowo przełączane w tryb uczenia (`model.train()`).

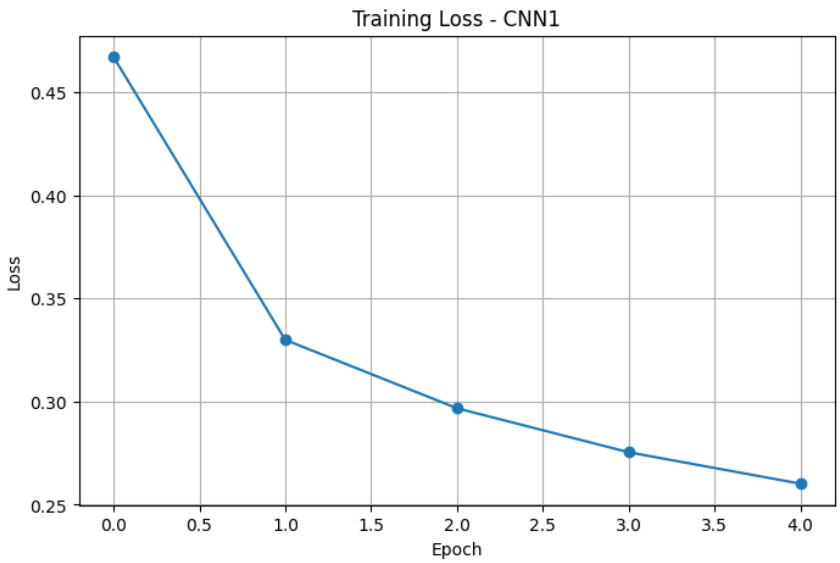
2.5 Ewaluacja i wizualizacja wyników

W fazie testowania modele przełączano w odpowiedni tryb ewaluacji (`model.eval()`) oraz wyłączano obliczanie gradientów (`torch.no_grad()`), co gwarantowało miarodajność wyników. Ocenę jakości modeli przeprowadzono z wykorzystaniem:

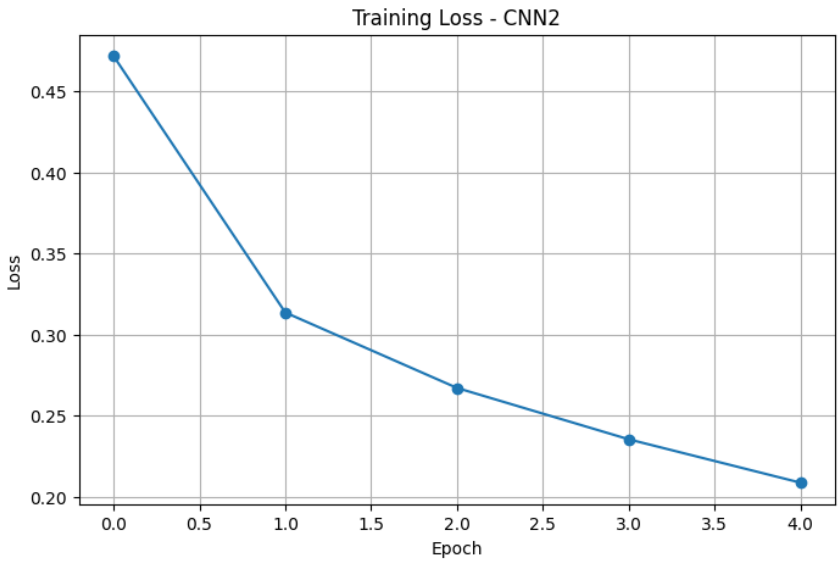
- Wykresów funkcji strat w zależności od epoki.
- Macierzy pomyłek (ang. *confusion matrix*) generowanych za pomocą biblioteki `scikit-learn` i wizualizowanych w postaci czytelnych map cieplnych (ang. *heatmap*) przy użyciu biblioteki `seaborn`.
- Ekstrakcji oraz wizualizacji przykładowych obrazów, które zostały błędnie zaklasyfikowane, wraz z podaniem etykiety rzeczywistej oraz przewidzianej przez sieć.

2.6 Analiza krzywych uczenia (funkcji strat)

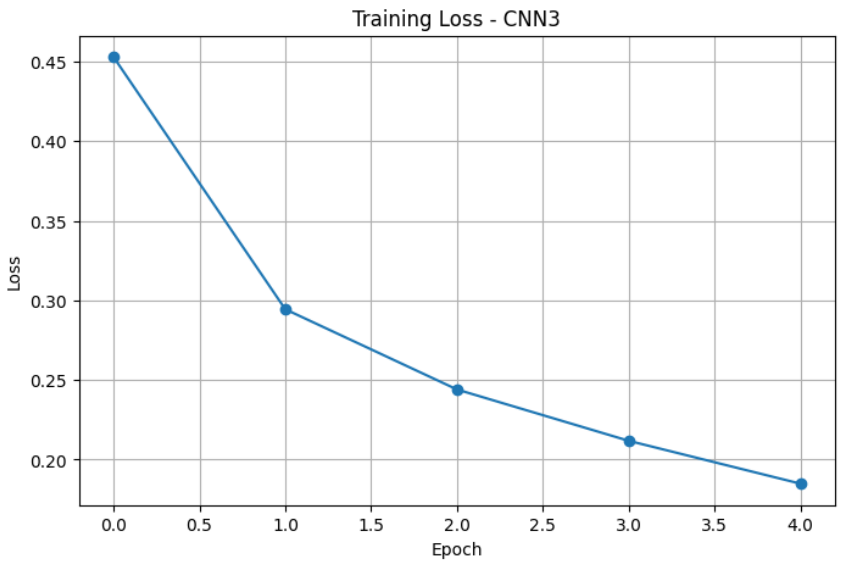
W celu weryfikacji poprawności i stabilności procesu treningowego, dla każdej z architektur wygenerowano wykresy przedstawiające spadek wartości funkcji strat (ang. *training loss*) w kolejnych epokach uczenia.



Rysunek 1: Spadek funkcji strat dla CNN1



Rysunek 2: Spadek funkcji strat dla CNN2

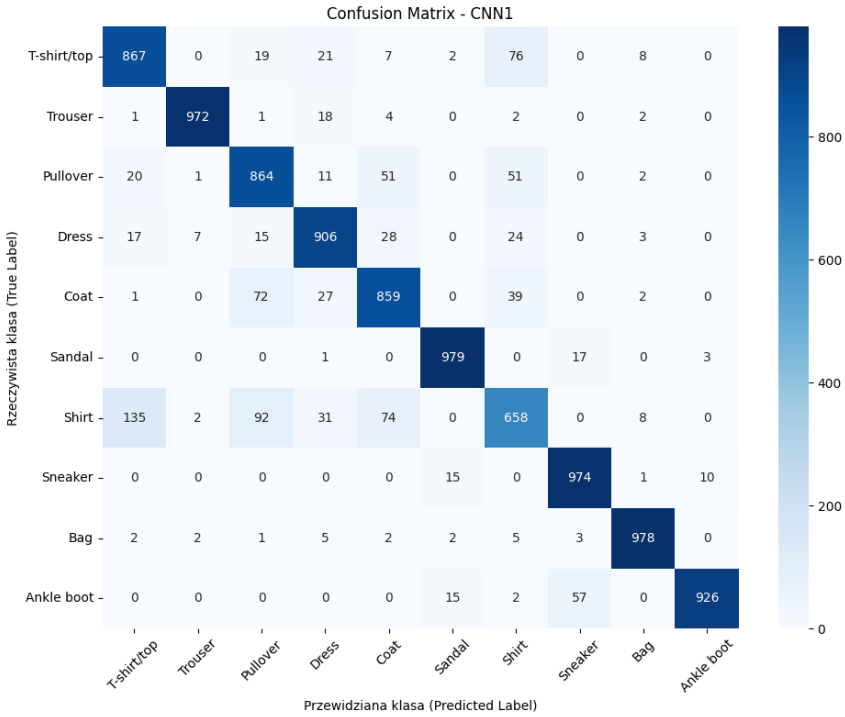


Rysunek 3: Spadek funkcji strat dla CNN3

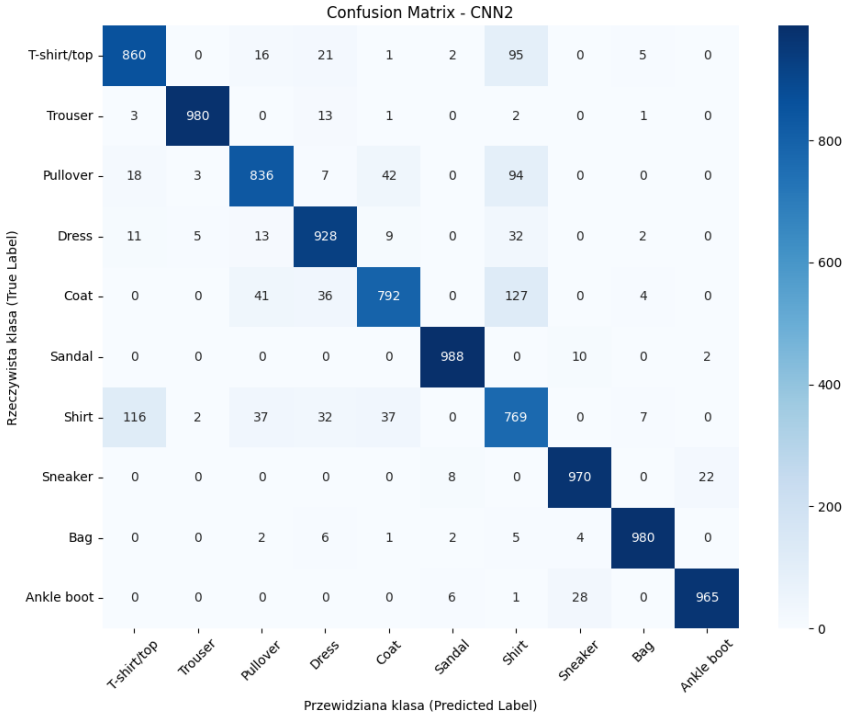
Na podstawie analizy przedstawionych wykresów można zauważyć, że:

- **Prawidłowa zbieżność modeli:** We wszystkich trzech przypadkach funkcja strat maleje w sposób płynny i stabilny. Świadczy to o odpowiednim doborze algorytmu optymalizacyjnego (Adam) oraz prawidłowym przebiegu procesu minimalizacji błędu.
- **Wpływ pojemności sieci na błąd treningowy:** Wartość końcowa funkcji strat po 5 epokach zależy od złożoności modelu. Najprostsza architektura, CNN1, osiągnęła najwyższą końcową wartość straty (około 0.26). Dodanie drugiej warstwy konwolucyjnej w CNN2 pozwoliło na lepszą minimalizację błędu (spadek do około 0.21).
- **Zjawisko dopasowania (ang. *fitting*):** Model CNN3, posiadający najbardziej rozbudowaną warstwę w pełni połączoną (256 neuronów), osiągnął najniższą wartość funkcji strat na zbiorze treningowym (poniżej 0.19). Jednakże, zestawiając ten wynik z przeanalizowanymi macierzami pomyłek dla zbioru testowego, można wywnioskować, że głębsza minimalizacja błędu treningowego w CNN3 nie przełożyła się na proporcjonalnie wyższą dokładność klasyfikacji nowych danych. Może to sugerować początkowe stadium przeuczenia (ang. *overfitting*) ze względu na zwiększoną pojemność modelu.

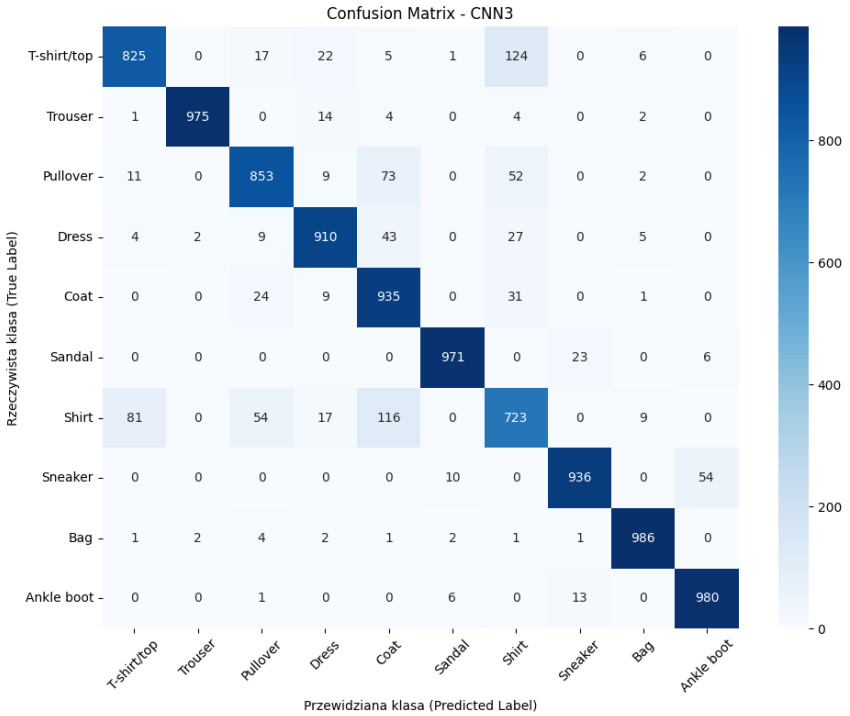
2.7 Macierze pomyłek



Rysunek 4: Wyniki dla CNN1



Rysunek 5: Wyniki dla CNN2



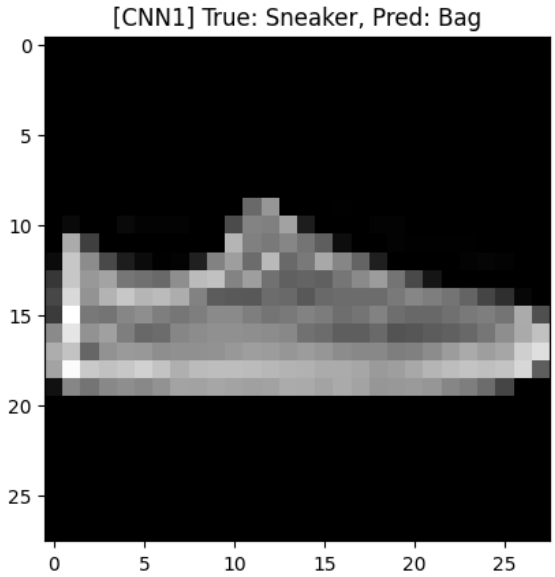
Rysunek 6: Wyniki dla CNN3

Na podstawie wygenerowanych macierzy pomyłek (ang. *confusion matrix*) dla trzech badanych architektur sieci konwolucyjnych (CNN1, CNN2, CNN3) można sformułować następujące wnioski:

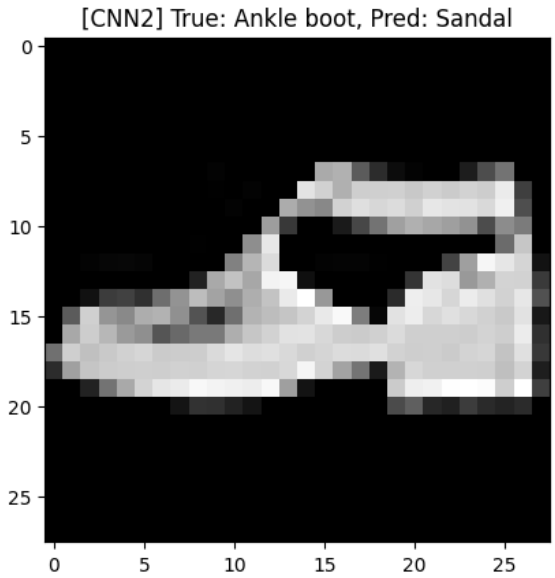
- **Wysoka skuteczność dla klas o unikalnych cechach:** Wszystkie trzy modele wykazują bardzo wysoką dokładność (często przekraczającą 95-97%) w rozpoznawaniu obiektów o specyficznych, wyraźnych kształtach. Klasy takie jak *Trouser* (spodnie), *Bag* (torba) oraz obuwie (*Sandal*, *Sneaker*, *Ankle boot*) są rzadko mylone z innymi częściami garderoby.
- **Problematyczne klasy (odzież wierzchnia):** Największym wyzwaniem dla sieci jest rozróżnienie ubrań górnych: *Shirt* (koszula), *T-shirt/top* (koszulka), *Pullover* (sweter) oraz *Coat* (płaszcz). Wynika to z faktu, że na obrazach o niskiej rozdzielczości (28x28 pikseli) sylwetki tych ubrań są do siebie bardzo zbliżone. Klasą o najniższej skuteczności we wszystkich modelach jest *Shirt*.
- **Wpływ architektury na wyniki:**
  - **CNN1:** Najprostsza sieć (tylko jedna warstwa konwolucyjna) poradziła sobie najgorzej z trudnymi klasami, prawidłowo klasyfikując jedynie 658 koszul (*Shirt*), często myląc je z koszulkami i swetrami.
  - **CNN2:** Zastosowanie drugiej warstwy konwolucyjnej (32 i 64 filtry) znacząco poprawiło zdolność modelu do ekstrakcji bardziej złożonych cech przestrzennych. Skuteczność dla klasy *Shirt* wzrosła do 769 poprawnych predykcji, a ogólna liczba błędów w klasach odzieży wierzchniej spadła.
  - **CNN3:** Zwiększenie liczby neuronów w ukrytej warstwie w pełni połączonej (z 128 do 256 neuronów) nie przyniosło dalszej istotnej poprawy względem CNN2. Wskazuje to, że w przypadku analizy obrazów kluczowe

jest pogłębianie warstw konwolucyjnych (ekstrakcja cech), a nadmierne rozbudowywanie warstw gęstych bez silnej regularyzacji nie przekłada się bezpośrednio na lepszą generalizację modelu.

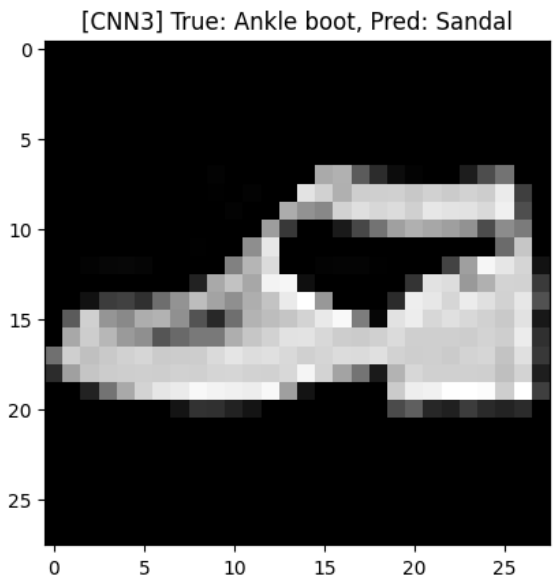
2.8 Przykłady błędów klasyfikacji



Rysunek 7: CNN1 źle klasyfikuje "Sneaker"



Rysunek 8: CNN2 źle klasyfikuje "Ankle boot"



Rysunek 9: CNN3 źle klasyfikuje "Ankle boot"

3 Zadanie 2

3.1 Opis zadania:

Kolejnym etapem laboratorium było zbadanie wpływu akceleracji sprzętowej na czas uczenia konwolucyjnej sieci neuronowej. Dla jednej wybranej architektury (w tym przypadku modelu CNN2) należało przeprowadzić proces treningu dwukrotnie: najpierw z wykorzystaniem standardowego procesora głównego (CPU), a następnie przy użyciu jednostki przetwarzania graficznego (GPU).

3.2 Proces realizacji eksperymentu (CPU vs GPU)

Proces porównania czasów uczenia został zrealizowany w następujących krokach:

- **Inicjalizacja środowiska:** Sprawdzono dostępność akceleratora graficznego za pomocą funkcji `torch.cuda.is_available()` i zdefiniowano odpowiednie urządzenia obliczeniowe (`cpu` oraz `cuda`).

```
    #Check if GPU is available
import torch
print(torch.cuda.is_available())  # should be True
```

- **Funkcja pomiarowa:** Zdefiniowano dedykowaną funkcję `train_with_time`, która realizuje standardową pętlę uczenia (5 epok) z jednoczesnym pomiarem czasu wykonania przy użyciu biblioteki `time`.

```
def train_with_time(model, device):
    model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    start = time.time()
```

```
    for epoch in range(5):
        for images, labels in train_loader:
            images = images.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

    end = time.time()

    return end - start
```

- **Zarządzanie pamięcią (Transfer danych):** Zadbano o poprawne przeniesienie zarówno wag modelu (`model.to(device)`), jak i każdego wsadu danych treningowych (`images.to(device)`, `labels.to(device)`) na docelowe urządzenie obliczeniowe tuż przed wykonaniem operacji.

3.3 Wyniki i wnioski z porównania CPU i GPU

Po przeprowadzeniu procesu uczenia wybranej architektury (CNN2) przez 5 epok na dwóch różnych jednostkach obliczeniowych, uzyskano następujące czasy wykonania:

- **Czas uczenia na CPU:** 181.70 sekund
- **Czas uczenia na GPU (NVIDIA T4):** 76.51 sekund
- **Przyspieszenie (ang. *Speedup*):** 2.37x

Wnioski:
Wyniki eksperymentu jednoznacznie potwierdzają przewagę jednostek graficznych (GPU) nad tradycyjnymi procesorami głównymi (CPU) w kontekście uczenia głębokich sieci neuronowych. Zastosowanie akceleratora sprzętowego pozwoliło na skrócenie czasu treningu ponad dwukrotnie (przyspieszenie rzędu 2.37x).

Znacząca różnica w wydajności wynika z samej architektury tych układów. Procesory graficzne (GPU) posiadają tysiące mniejszych rdzeni obliczeniowych, które umożliwiają wysoce zrównoleglone wykonywanie operacji na macierzach i tensorach. Ponieważ proces uczenia konwolucyjnych sieci neuronowych (CNN) polega w głównej mierze na masowych mnożeniach macierzy, wykorzystanie GPU jest kluczowe dla zapewnienia optymalnej wydajności. W przypadku pracy z obrazami o wyższej rozdzielczości lub jeszcze bardziej skomplikowanymi sieciami, zysk wydajnościowy byłby jeszcze bardziej drastyczny.